Project on Tomographic Reconstruction
SF2529 HT26

Célia Manioc

1 Introduction

This report is a technical report of the results that I obtained in the project "Project on Tomographic Reconstruction" for the class SF2529-HT26.

1.1 Problem

Let $x^\dagger$ be the image to reconstruct and A the ray transform. The data are $y = A(x^\dagger) + e$ and I call $\delta = \|e\|/\|A(x^\dagger)\|$ the relative noise level. Unless said otherwise, I used $\delta = 0.01$ as asked in the project description.

1.2 Method

To solve this ill-posed problem, I used the Huber regularisation:

$$\min_x \ \frac{1}{2}\|A(x) - y\|_2^2 + \lambda H_\epsilon(\nabla x), \qquad H_\epsilon(z) = \sum_i h_\epsilon(|z_i|), \tag{1}$$

where $h_\epsilon(t) = t^2/(2\epsilon)$ if $|t| \leq \epsilon$ and $|t| - \epsilon/2$ otherwise, and ∇ is a finite difference gradient. For a small ϵ, H_ϵ is a differentiable approximation of the ℓ^1-norm, so the reconstructions have regions of almost constant values separated by sharp borders.

To minimise this functional I used the steepest descent method with a backtracking line search. At each iteration, $x_{k+1} = x_k + t_k r_k$ with $r_k = -\nabla f(x_k)$, and the step t_k is found by starting from a value α and reducing it until the functional decreases enough. The solver stops when the gradient is smaller than a tolerance, when the maximum number of iterations is reached, or when the line search fails (see section *Failure of the solver depending on the line search parameters* 7.2).

The gradient of the functional is Lipschitz with a constant smaller than $\|A\|^2 + \lambda\|\nabla\|^2/\epsilon$. That is why a small ϵ forces a small step size and why I tried to keep the biggest ϵ that gave a good reconstruction.

2 My familiarization with the problem and the use of ODL

At the beginning of the project I tried to familiarize myself with ODL and its use to solve inverse problems. Notably, I tried to change a lot of the parameters given in the project description. Here are the following observations that I made regarding its use :

- I tried to use lower resolution but it wasn't possible to make in depth visual analysis with them.
- I made the noise level δ vary between 10^{-6} and 10^{1}, while the differences were not really visible to the naked eye, it changed the optimal values that I found, visually, for the parameters λ and ϵ while doing my test on the reconstructed images. For the noise level asked, $\delta = 0.01$ I found that the optimal parameters were $\lambda = 1.0$ and $\epsilon = 0.001$ while for $\delta \approx 2 \cdot 10^{-6}$ there were $\lambda = 0.01$ and $\epsilon = 0.01$. The code and my visual criteria are detailed in the section *Choice of λ and ϵ* [3].
- I tried to modify the tolerance but since I never achieved a gradient very low it didn't produce interesting results. I suppose that it was because I was using float32 numbers.

3 Choice of λ and ϵ

3.1 Visual analysis

I wanted to test different combinations of λ and ϵ in order to study their impact on the reconstruction. Some combinations caused the solver to fail after a lot of iterations (because of the use of float32), so I used a "try block" to handle these cases and continue the tests. The failure is explained more in depth in the section *Failure of the solver depending on the line search* [7.2]. I tested multiple k and tolerance as I wanted to go at the maximum iterations available to have the most optimized reconstruction possible for each combination.

For each combination, I evaluated both the final gradient and the reconstructed image. This was useful because a lower gradient did not always correspond to a better reconstruction: in some cases, the reconstructed image contained more noise or was less suitable for interpretation despite having a relatively low final gradient.

I also displayed the exact solution and the naive reconstruction to provide reference points and make the comparison between the different results more efficient.

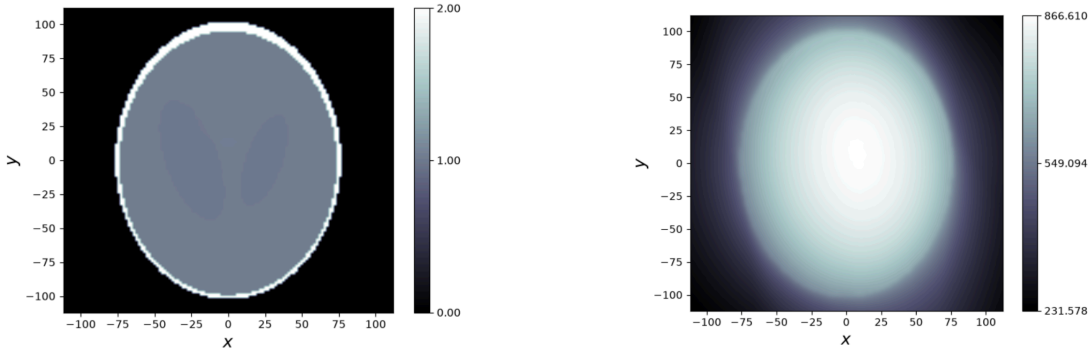
The result of all of my tests is that $\epsilon = 0.001$ is generally a good value : most of the time it is the first reconstruction without visible noise. The gradient is also relatively good.

On visual criteria, the optimal combination is ($\lambda = 1.0$ and $\epsilon = 10^{-3}$), on my longer test it was better than above. ($\lambda = 1.0$ and $\epsilon = 10^{-2}$) and ($\lambda = 1.0$ and $\epsilon = 10^{-4}$) (with

more iterations than on the code handed over) were also really good. For $\epsilon = 10^{-2}$, the reconstruction was a bit noisy and for $\epsilon = 10^{-4}$ it was a bit too smooth/blurry. I also tried to keep the biggest ϵ that returned a good result given the fact that the smaller ϵ is the smaller the search step must be.

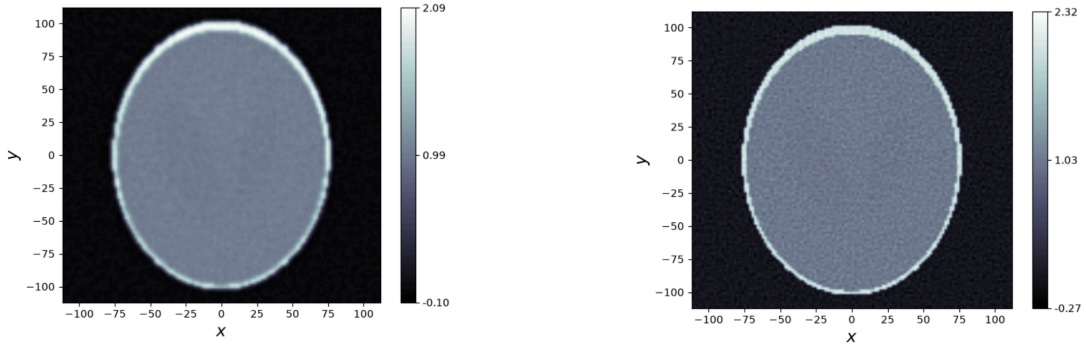
In general, I observed that when λ decreases, the global noise of the reconstructed image goes up and that when ϵ goes down, it smooths the global image and either reduce the noise or make the reconstructed image a bit blurry.

Below the exact data and the naive reconstruction of the 2D phantom of Shepp-Logan for comparison purposes.



(a) Exact data (b) Naive reconstruction

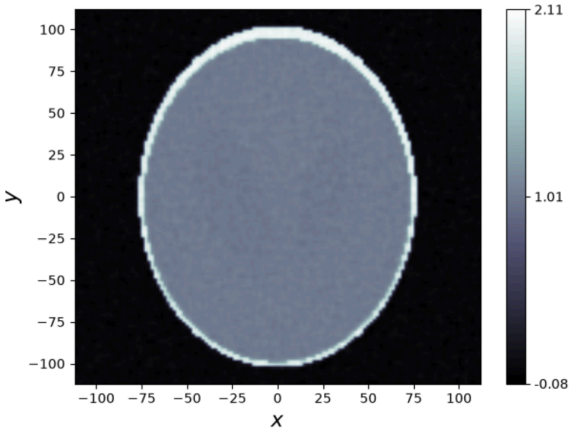
Figure 1: 2D phantom of Shepp-Logan



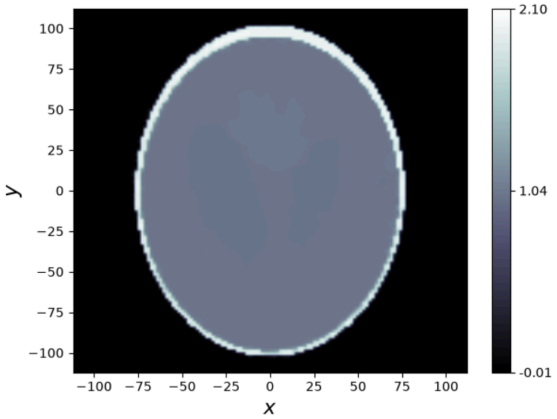
(a) $\lambda = 100$ and $\epsilon = 10$ (b) $\lambda = 1$ and $\epsilon = 10$

Figure 2: Evolution of the reconstruction for different values of λ

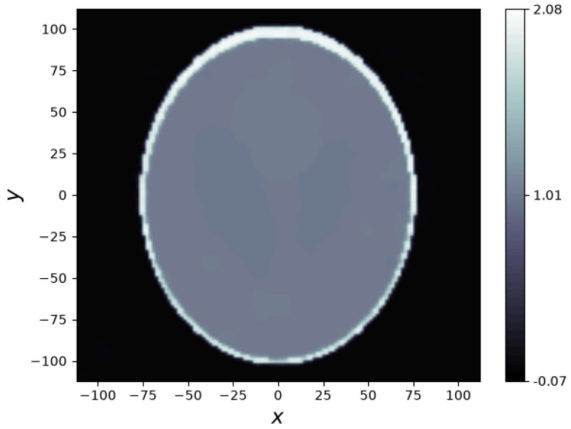
In the Figure 2, we can see that the noise is slightly greater in the second reconstruction. It is coherent because using a smaller regularisation parameter reduces its effect. We can also see a bit of over-regularisation with the first reconstruction being blurry.



(a) $\lambda = 1$ and $\epsilon = 10^{-1}$



(b) $\lambda = 1$ and $\epsilon = 10^{-3}$



(c) $\lambda = 1$ and $\epsilon = 10^{-5}$

Figure 3: Evolution of the reconstruction for different values of ϵ

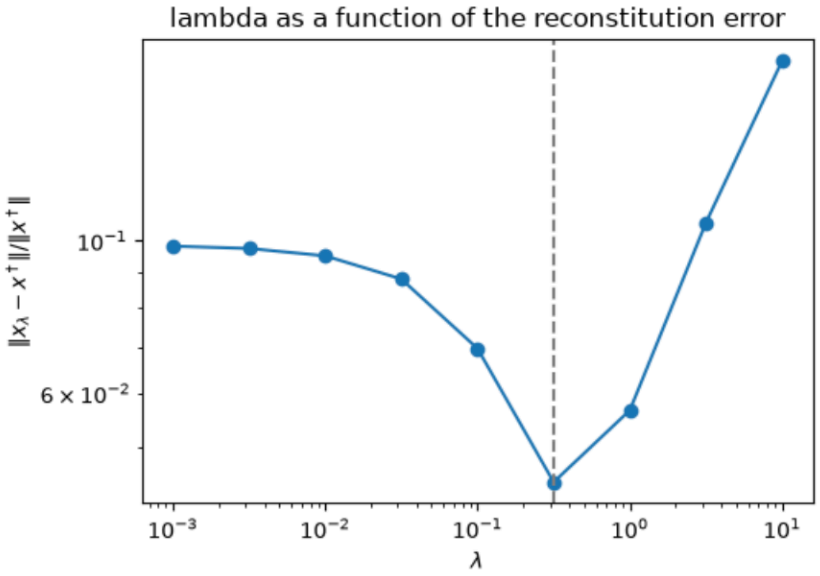
In the Figure 3, we can see that the noise decreases as ϵ decreases but that the reconstruction becomes a bit smoother. On the 3rd image we can barely see the ellipsoids inside of the object.

Another observation is that the value of ϵ doesn't affect the reconstruction as much when λ is very small. That is coherent because
$\lambda \to 0 \implies \lambda H_\epsilon(\nabla x) \to 0 \implies \min_x \ \frac{1}{2}\|A(x) - y\|_2^2 + \lambda H_\epsilon(\nabla x) \to \min_x \ \frac{1}{2}\|A(x) - y\|_2^2$.

3.2 Search of the optimal λ

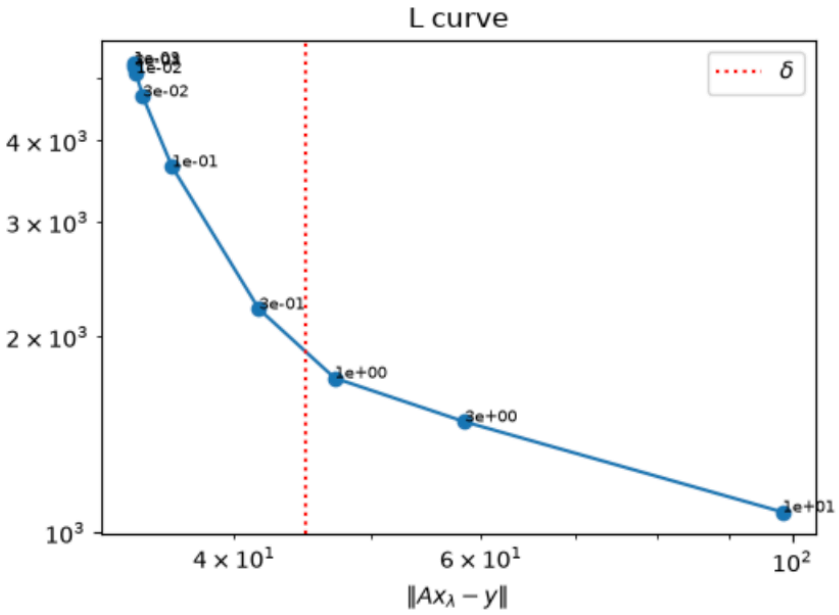
To search for the optimal λ, I implemented two methods.

First, I printed the reconstruction error for multiple values of λ to find the one minimising it. It gave me an excellent approximation of the optimal λ however since it was only possible because I knew the exact data we won't be able to use it on the 3D CT data.



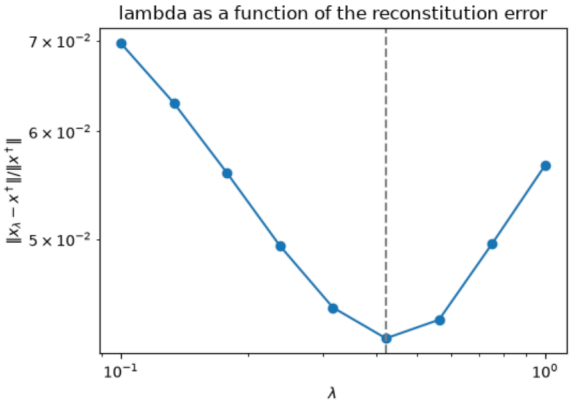
As observed in the visual reconstructions, the optimal value of λ is close to 1. In particular, $\lambda = 1$ is the first value (in the decreasing order) for which the error is lower than 0.1. Although it does not produce the lowest relative error, this value is considered acceptable since it is of the same order as the optimal value. The goal being to reduce the reconstruction error (noise error + regularisation error).

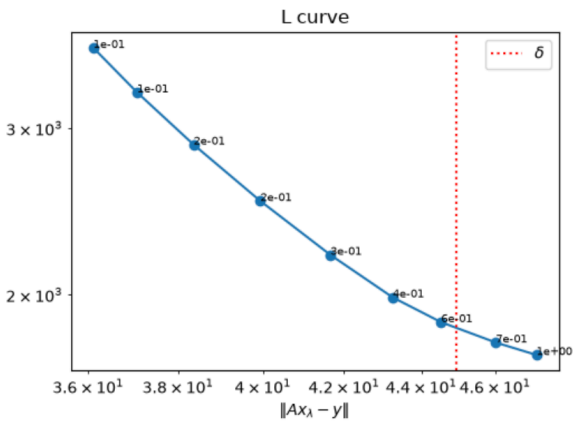
The second method implemented was the L-curve. I was then able to compare the results of the two methods to verify that they are coherent. See the graphs below.



Without knowing the real phantom, we see that the "elbow" of the L-curve is between 0.03 and 1. That is coherent with the result observed in the first graph and visually. That means that we can consider both of the results equivalent to a certain degree and use the L-curve for the 3D problem.

I ran the codes with smaller intervals, closer to the minimum found previously and obtained a slight difference between the two methods.





For the first graph, the optimal λ is close to 0.42 while for the L-curve it's closer to 0.6. That was expected for a more in depth analysis. That doesn't mean that the results observed on the L-curve are less reliable just less precise than the minimim of the reconstruction error. Especially given the fact that the difference in the reconstruction error of the minimum observed on the 1st and 2nd graph is less than 0.002.

Approximation of the elbow of the L-curve: $\lambda = 5.6e-01$ error=0.0435

Approximation of the λ minimising the reconstruction error: $\lambda = 4.2e-01$ error=0.0421

Finally, the results above indicate that the optimal λ is approximately 0.56 according to the L-curve and is included between $4 \cdot 10^{-2}$ and $4.5 \cdot 10^{-2}$ according to the relative error. Those values are still very close to the 1 found by visual analysis.

For the following tests I made λ vary between 0.01, 0.5, 0.67 and 1.

4 Behaviour of the solver in 2D

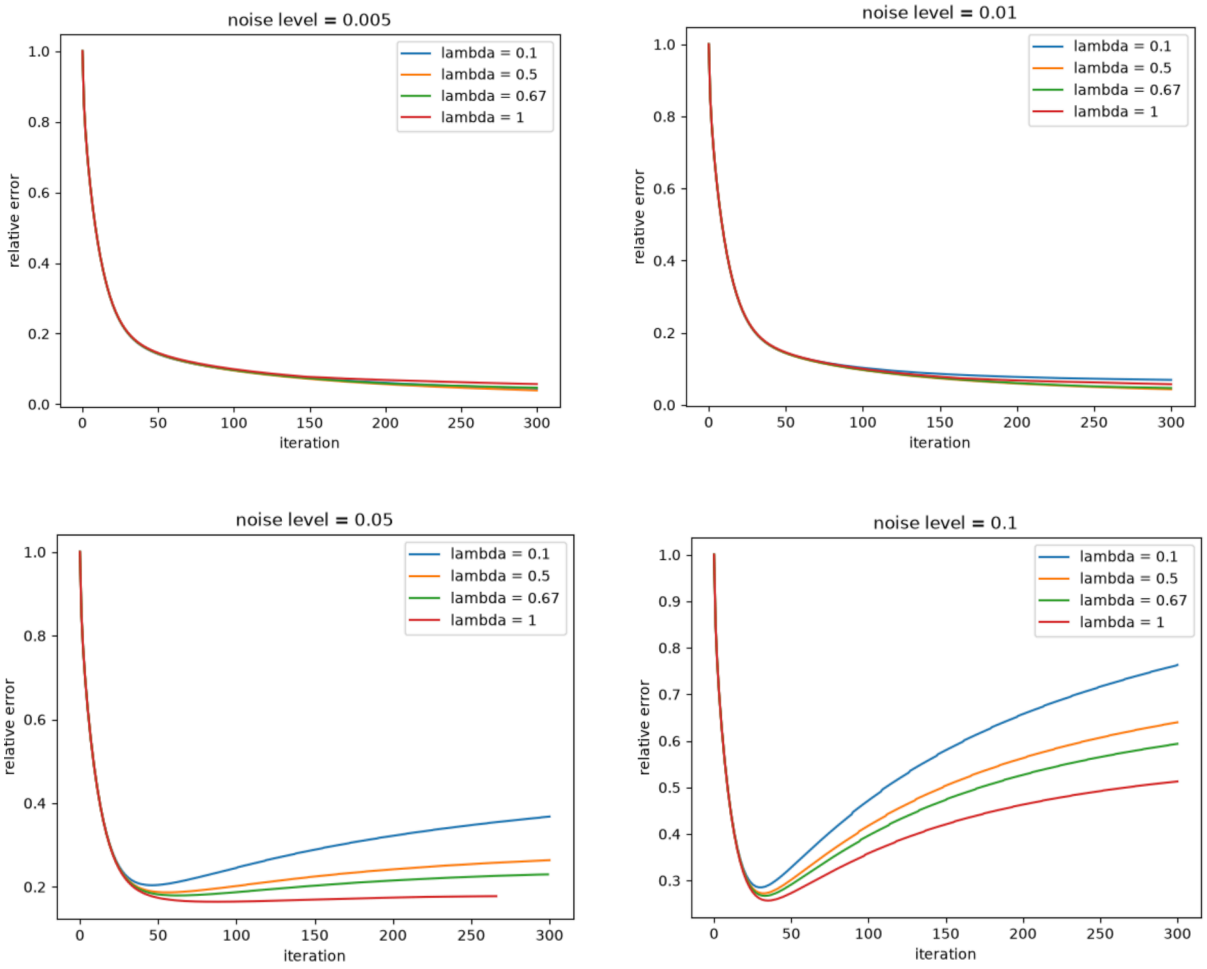


Figure 5: 2D reconstruction error as a function of the number of iterations.

I ran the solver for 300 iterations for four noise levels and for the values of λ found above. For each iteration I plotted the relative error between the reconstruction and the exact phantom.

First, I observed that for all the noise levels the curves are the same during the first 30 iterations or so. At the beginning the solver is only fitting the data, so the regularisation has no effect. The differences appear after.

For a noise level of 0.005, all the values of λ give almost the same result. The error decreases quickly and then more slowly, without any semi-convergence. At 300 iterations $\lambda = 1$ is a bit higher.

For a noise level of 0.01 it is similar. The best final error is obtained with $\lambda = 0.5$ and $\lambda = 0.67$ and the worst with $\lambda = 0.1$.

For a noise level of 0.05, the semi-convergence becomes visible. For $\lambda = 0.1$ the error is minimal after about 50 iterations and then goes up. When λ increases, the minimum is reached later and the error goes up less. For $\lambda = 1$ the minimum is around 100 iterations and the curve is almost flat after. The curve of $\lambda = 1$ stops a bit before 300 iterations because the solver stopped.

For a noise level of 0.1, the semi-convergence is even stronger. For all the values of λ the minimum is reached after 30 to 35 iterations and is almost the same, and then the error goes up a lot. After 300 iterations $\lambda = 1$ is still the best but the semi-convergence is still present.

In conclusion, I observed that the regularisation does not change much the best error that can be reached but it limits how fast the error goes up after, so the number of iterations is less critical. That is coherent with the project description saying that the regularisation only partially takes care of the semi-convergence. For low noise levels the choice of λ has a small effect while a bigger λ is better when the noise level increases.

5 3D reconstruction

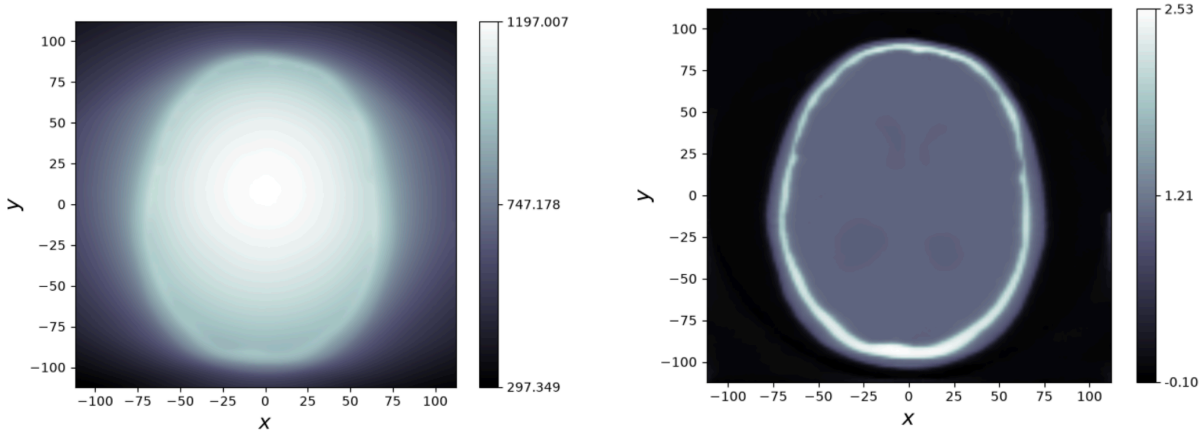
5.1 Reconstruction of 3D data

5.1.1 Test on 3D Shepp-Logan

First of all I tried to reconstruct the 3D phantom of Shepp-Logan to verify that my reconstruction function still worked for 3D problem and returned a good reconstruction. I achieved similar images as for the 2D phantom so I moved to the data of the 3D-CT.

5.1.2 3D CT

The object reconstructed is the one below :

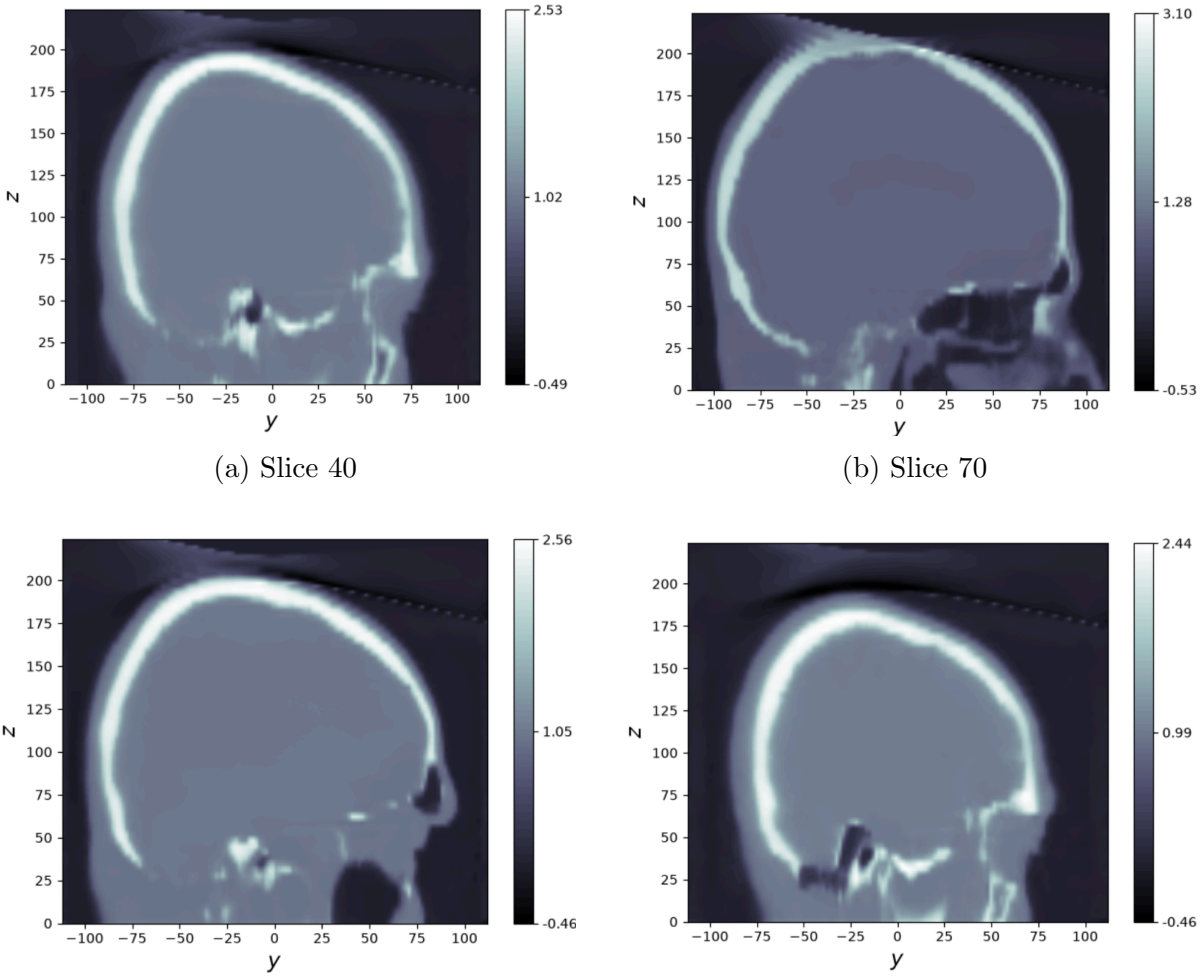


(a) Naive reconstruction (b) regularised reconstruction

Figure 6: Reconstructions of the 3D-CT object

On those reconstructions we can see the shape of the head and the skull around it. For the regularised reconstruction the parameters were the following : $\lambda = 1.0$, $\epsilon = 10^{-3}$ and $k = 30$.

5.1.3 Some 2D slices of the reconstruction



(a) Slice 40

(b) Slice 70

(c) Slice 80

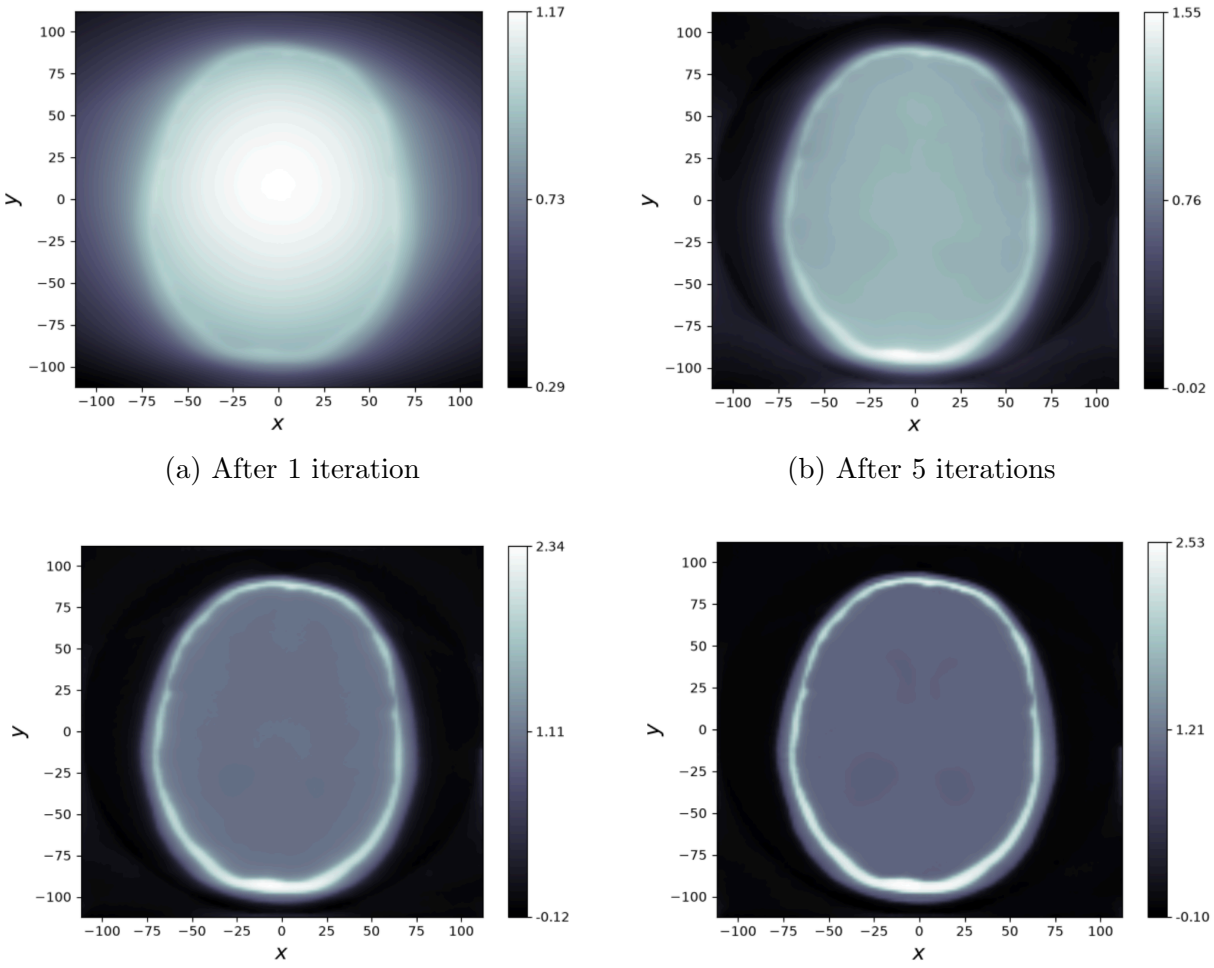
(d) Slice 90

Figure 7: 2D slices of the reconstructed object

Those slices show another orientation of the head.

5.2 Variation of the reconstruction parameters

5.2.1 Impact of the number of iterations



(a) After 1 iteration

(b) After 5 iterations

(c) After 15 iterations

(d) After 30 iterations

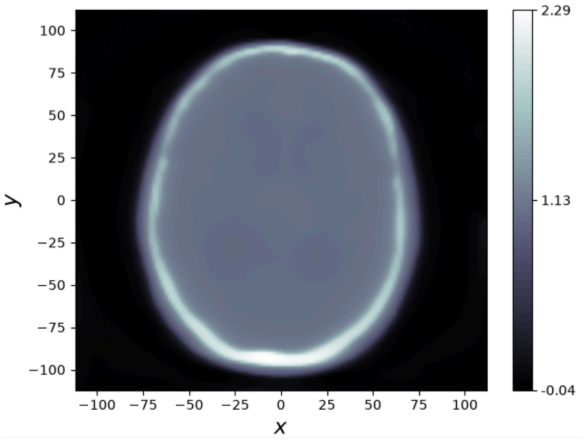
Figure 8: Evolution of the reconstructed object ($\lambda = 1$ and $\epsilon = 10^{-3}$) after different numbers of iterations

At the first iteration the reconstruction is a smooth blob, similar to the naive reconstruction. After 5 iterations the skull starts to appear but there is still a dark halo around the object and the inside is too bright. After 15 iterations the skull is sharp and the soft tissues around it are visible. Between 15 and 30 iterations the changes are small : the contours of the soft tissues become a bit cleaner and the values stabilise. That is coherent because at each iteration the gradient descent approaches the minimum of f. I only went to 30 iterations because each one was taking a lot of time and with 30 iterations we already have a great result.

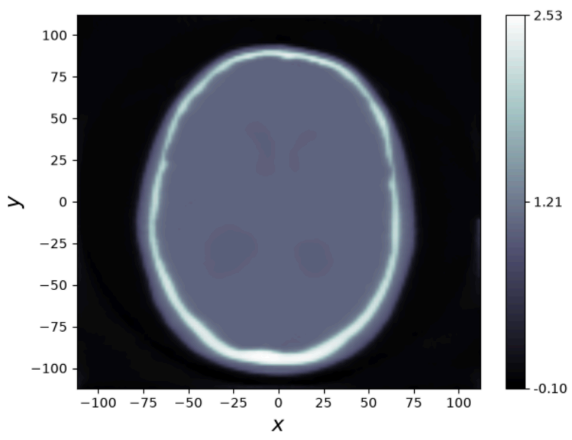
5.2.2 Effect of the regularisation parameter λ

I observed the same evolution of the reconstructions by varying the λ as in the 2D case. See the section *Visual analysis* [3]

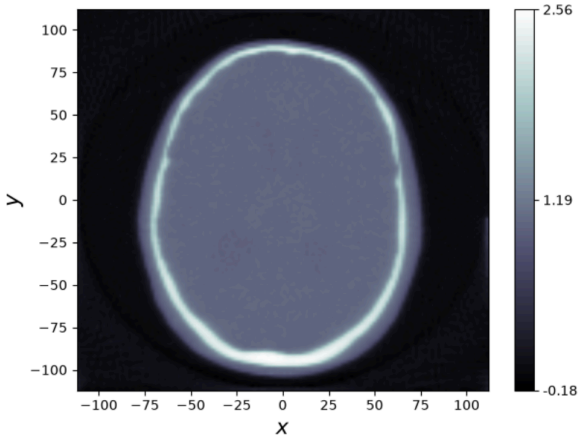
Below are some reconstructed images after 30 iterations with $\epsilon = 10^{-3}$:



(a) $\lambda = 10.0$



(b) $\lambda = 1.0$



(c) $\lambda = 10^{-3}$

Figure 9: Evolution of the reconstruction for different values of λ on the 3D CT dataset

Visually, for $\lambda = 10.0$, the reconstruction is a bit blurrier than for $\lambda = 1.0$. That is probably due to a slight over-regularisation. The reconstruction for $\lambda = 10^{-3}$ contains a little bit of noise.

On the L-curve associated to the 3D CT dataset we can also see that the elbow of the curve is still close to $\lambda = 1.0$ which is the reason why I kept using that value throughout the reconstructions.

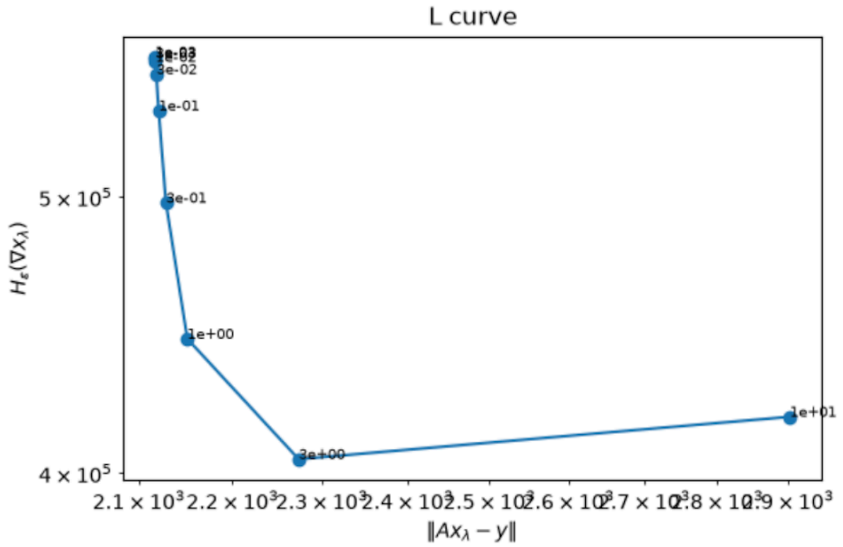
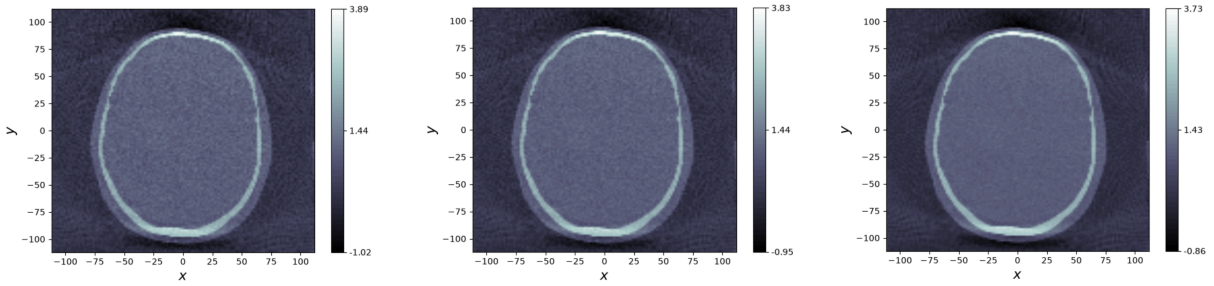


Figure 10: L-curve associated to the 3D-CT dataset

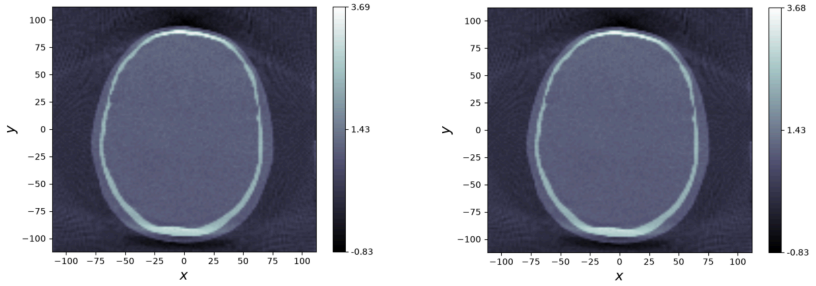
To obtain that graph I did reconstructions of 20 iterations maximum because of time constraints.

6 Filtered Back Projection

For the Filtered Back Projection I used the *fbp_op* method of ODL on the noisy data of the 3D CT. I compared the filters Ram-Lak, Shepp-Logan, Cosine, Hamming and Hann.



(a) Ram-Lak (b) Shepp-Logan (c) Cosine

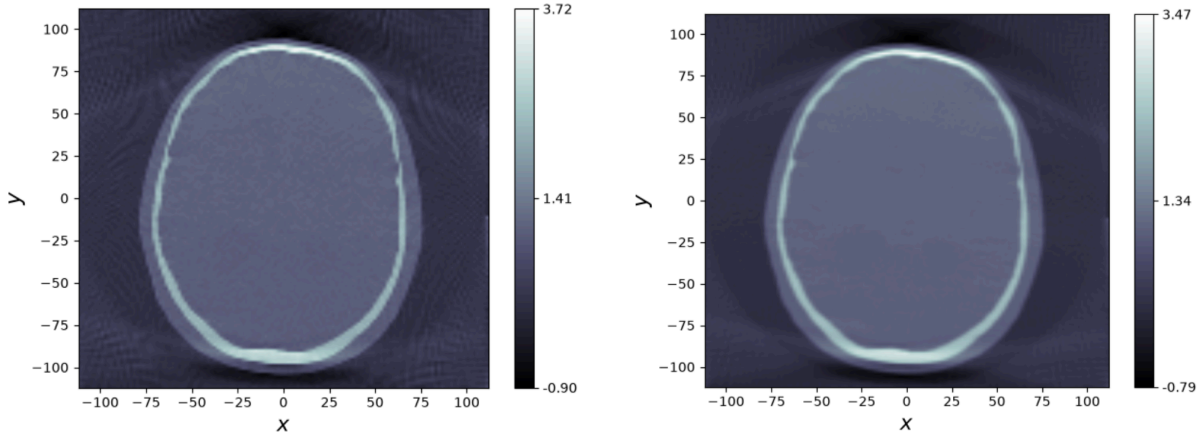


(d) Hamming (e) Hann

Figure 11: Filtered Back Projection of the 3D CT with the default frequency scaling

The five reconstructions are quite similar. They are less blurry than the regularised reconstruction but the noise is still present inside the object. The Ram-Lak filter is the noisiest and the Hamming and Hann filters the least noisy, which is coherent with the fact that the Ram-Lak filter does not reduce the high frequencies. We can also still see the ellipsoidal residual of the original data outside of the object, and the values outside of the object are noisy and negative (down to about -1).

The parameter *frequency_scaling* is the relative cut-off frequency of the filter, so a smaller value gives a smoother image. To reduce the noise I decided to lower the frequency scaling to 0.3 for the Ram-Lak and the Hann filters. I chose 0.3 because it was the first value to, visually, give me a satisfactory reconstruction. I also tested 0.5, 0.7 and 0.8.



(a) Ram-Lak, scaling 0.3

(b) Hann, scaling 0.3

Figure 12: Comparison of the Filtered Back Projection with a low frequency scaling

With the Ram-Lak filter we have much less noise but small oscillations are still visible outside of the object and the little circles inside of it aren't visible. With the Hann filter the reconstruction is much better: the inside of the object is smooth and the little circles are a little bit visible (barely). I think that the Hann filter is smoother because it reduces the frequencies progressively before the cut-off, while the Ram-Lak filter keeps them until the cut-off. In both cases we can still see the outline of the residuals outside of the object but given the fact that it doesn't perturb the visualisation of the object itself, I think that it should be fine.

Compared to the regularised reconstruction, the Filtered Back Projection with the Hann filter is less blurry, the values in the skull are higher but the little circles inside are less visible. There is also the fact that the regularised reconstruction has a cleaner background, the values outside of the object are close to 0 and it does not have the oscillations. The Filtered Back Projection also needs only one pass while the regularised problem needs 30 iterations, each one taking a lot of time. So for this noise level, I think that the Filtered Back Projection with a Hann filter and a low frequency scaling is a good alternative, and that the regularisation is better if we want a cleaner or more precise image.

7 Issues and difficulties that I encountered

7.1 The implementation of the problem itself

At first I used one of the example codes in the *Source code for odl.functionals.functional*[1] in the ODL Documentation. It was a great practice exercise I think to understand better the objects with which I was working. However, by defining the functional that way and including the evaluation of $A(x)$ and $grad(x)$ at each step I had some imprecision (I think because of float32) that blocked the use of the BacktrackingLineSearch and caused me some errors. To fix this, I constructed the functional f using the code examples of the project description. The code is described below.

```
l = 1
epsilon = 1/10**3
l2 = odl.functionals.L2NormSquared(forward.range)
data_fit = l2.translated(data)

grad = odl.Gradient(space)
Heps = odl.functionals.Huber(grad.range, epsilon)

regularisation = l * Heps * grad

f = 0.5* data_fit*forward + regularisation
```

My original functional isn't included in the code but can be found in the section *Implementation of f as a odl.functionals.functional* 10.1.

7.2 Failure of the solver depending on the line search parameters

I had an issue during my initial test in which I either didn't get a gradient close to 0 or the call to the solve method failed. That failure was an *AssertionError* inside the source code of the solver when checking if there were another point on which the norm of the gradient was lower than the one on the current point of iteration. At first, I was using the example given on the ODL documentation with its arguments and notably the *BacktrackingLineSearch* method called as below :

```
line_search = odl.solvers.BacktrackingLineSearch(f, alpha=1, max_num_iter=60)
```

Despite using the newer implementation of the functional I still had with some combinations of λ and ϵ errors. I tried using a constant step size when calling the solver. Notably, I tried using 10^{-3}, 10^{-4} and 10^{-5} with $\lambda = 1$ and $\epsilon = 10^{-3}$.

For 10^{-3} I still sometimes had the same error. For 10^{-4} and 10^{-5} the solver never failed but I needed close to ten times more iterations than before. Therefore, I used the

BacktrackingLineSearch and tried to put the same number of maximum iterations used in the call to the solver. That permanently solved the issue.

8 Conclusion

In this project I implemented a Huber regularised reconstruction method with the steepest descent solver and a backtracking line search, first on the 2D Shepp-Logan phantom and then on the 3D CT of a head. I also compared its results with the ones of the Filtered Back Projection.

For the parameters, I found that $\epsilon = 10^{-3}$ is a good compromise: it is the biggest ϵ without visible noise, and a smaller one gives a blurry image and forces a smaller step size. For λ, the value minimising the error in 2D is about 0.42 and the elbow of the L-curve is about 0.56, so the two methods are coherent. I kept $\lambda = 1$ for the 3D CT because it gave the best visual compromise between noise and smoothness. The optimal values also depend on the noise level: a smaller relative noise needs a smaller λ and a bigger relative noise needs a bigger one.

In 2D, I observed that the semi-convergence appears when the noise level increases, and that the regularisation limits it without removing it. In 3D the reconstruction was stable after about 30 iterations, which was enough to see the skull and the soft tissues around it.

The main difficulty was the solver failing because of float32 and the line search. I solved it by using the same maximum number of iterations in the line search and in the solver.

For the Filtered Back Projection, the filter and the frequency scaling change the noise a lot. The best result was obtained with the Hann filter and a frequency scaling of 0.3. It is sharper and much faster than the regularised reconstruction, but the regularised reconstruction has a cleaner background.

9 References

Documentation

[1] The ODL Contributors. *Source code for odl.functionals.functional*. URL: https://odl.readthedocs.io/_modules/odl/functionals/functional.html#FunctionalComp.__init__.

10 Appendix

10.1 Implementation of f as a odl.functionals.functional

```
class MyFunctional(odl.functionals.functional.Functional):

    """This functional implements the functional 4 of the project description
    and its gradient
    The implementation is based on the code described in the following example :
    https://odl.readthedocs.io/guide/functional_guide.html"""

    def __init__(self, space, operator, data, l_param = 1, epsilon = 1/10):
        # Sets `Operator.domain` to `space` and `Operator.range` to `space.field`
        super(MyFunctional, self).__init__(space)
        self.A = operator
        self.y = data
        self.l = l_param

        self.grad = odl.Gradient(operator.domain)

        self.Heps = odl.functionals.Huber(self.grad.range, epsilon)
        #since heps take in entry the grad of x

        self.l2 = odl.functionals.L2NormSquared(operator.range)

    def _call(self, x):
        """Evaluate the functional."""
        regularisation = self.l * self.Heps (self.grad(x))
        return 0.5 * (self.l2(self.A(x) - self.y) ) + regularisation

    @property
    def gradient(self): #to be able to use a pre-implemented solvers
        """The gradient operator."""

        # First we store the functional in a variable
        functional = self

        # The class corresponding to the gradient operator.
        class MyGradientOperator(odl.Operator):

            """Class implementing the gradient operator."""

            def __init__(self):
```

```
            """Initialize a new instance."""
            super(MyGradientOperator, self).__init__(
                domain=functional.domain, range=functional.domain)

        def _call(self, x):
            """Evaluate the gradient."""
            # Here we can access the store functional from a few lines
            # above
            huber = functional.Heps.gradient(functional.grad(x))
            reg = functional.l * functional.grad.adjoint(huber)
            return functional.A.adjoint(functional.A(x) - functional.y) + reg

    return MyGradientOperator()
```